

# CRUDL and MVC Part I

# What is **CRUDL**

When managing any list of entities, we commonly need 5 basic operation

- **C**reate or **Add** one to the list
- **R**ead or **Get** one from the list
- **U**ppdate or **Modify** one in the list
- **D**eleate or **Remove** one from the list
- **L**ist all items

# A Todo List as an Example



- Adding a todo to the list (**C**reate)
- Reading a todo's details from the list (**R**ead)
- Changing the todo's status to complete (**U**ppdate)
- Deleting a todo from the list (**D**eleate)
- Getting all todos in the list (**L**ist)

We would likely need to do the same with products, books, contacts or any other entity we manage in our app...

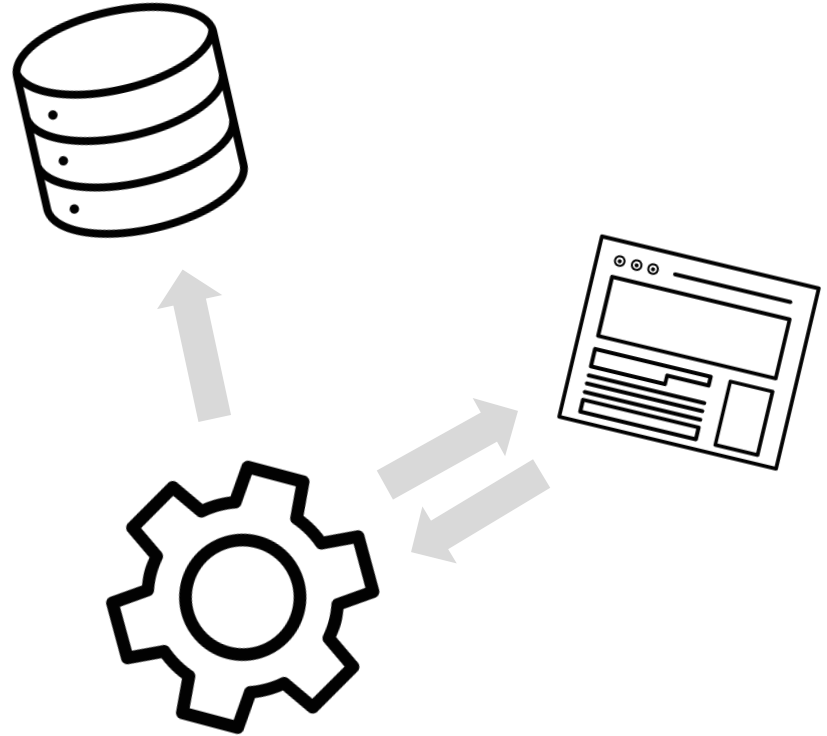
## **MVC: Model - View - Controller**

As our apps get more complex,  
we will find it beneficial to separate our code into layers  
with a well defined interface between them

As long as that interface remains unchanged  
We can freely change their internal implementations

# MVC: Model - View - Controller

- The **Model** layer manages all data related operations
- The **View** layer implements display and user interaction
- The **Controller** coordinates between the Model & View layers



# The Model Layer

- Responsible for managing our application's data
- This can be an array in memory, or any form of storage
- An interface to **CRUDL** operations is implemented as a **service**
- As long as this interface remains unchanged, the internal implementation of the service can be modified

# The Controller Layer

- Responds to events from the View (DOM)
- Renders the View (DOM)
- Calls a service which exposes an interface to all data related operations (Model)

# Separation of Concerns

- MVC provides good separation between the layers of our application.
- If the **View** layer is changed, the Model layer remains unaffected.
- If the **Model** layer is changed, the View layer remains unaffected
- In order for this to work - the Service **never** calls the controller

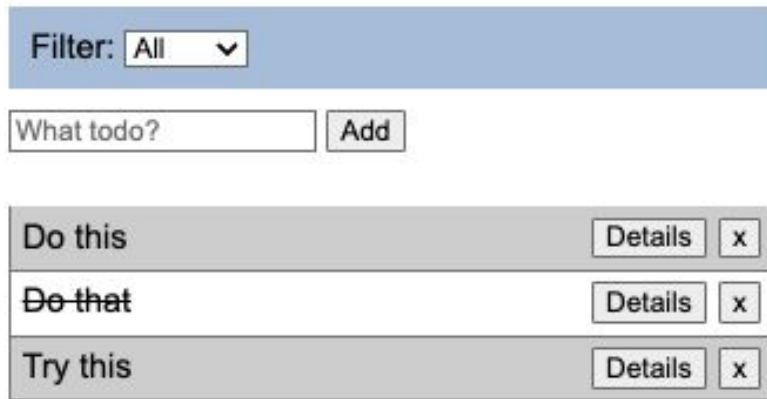


# Let's Build a Todo App

## Our Model

```
{  
  id: 't101',  
  txt: 'Do this',  
  isDone: false,  
}
```

## Our View



The screenshot shows a web application interface for a todo list. At the top, there is a blue header bar containing a 'Filter:' label and a dropdown menu currently set to 'All'. Below the header, there is a white input field with the placeholder text 'What todo?' and an 'Add' button to its right. The main content area displays a list of three todo items, each in a light gray row. The first item is 'Do this', the second is 'Do that', and the third is 'Try this'. Each item has a 'Details' button and a close button (represented by an 'x') to its right.

| Filter:                                       | All                                                                     |
|-----------------------------------------------|-------------------------------------------------------------------------|
| What todo? <input type="button" value="Add"/> |                                                                         |
| Do this                                       | <input type="button" value="Details"/> <input type="button" value="x"/> |
| Do that                                       | <input type="button" value="Details"/> <input type="button" value="x"/> |
| Try this                                      | <input type="button" value="Details"/> <input type="button" value="x"/> |